

Documentação Técnica — FoodClub

PI-VI 2026

DOCUMENTAÇÃO TÉCNICA

1. Identificação do Projeto
2. Visão Geral do Produto
3. Requisitos Funcionais e Não Funcionais
4. Arquitetura do Sistema
5. Modelo de Dados
6. APIs (Application Programming Interfaces)
7. Computação em Nuvem (Azure)
8. Qualidade e Testes de Software
9. Processamento de Linguagem Natural (PLN)
10. Integração entre Componentes
11. Riscos e Mitigações
12. Gestão de Sprints e Backlog
 - 12.1. Apreciação dos Sprints Realizados
 - 12.2. Itens do Product Backlog (PBI)
13. Anexos

DOCUMENTAÇÃO TÉCNICA

Modelo para o Projeto Integrador VI (PI-VI)

1. Identificação do Projeto

1.1. Nome do Projeto: FoodClub — Plataforma B2B de Alimentação Corporativa

1.2. Grupo/Equipe:

Nome	RA
Alinne Martins	3011392323003
Bruno Pasqual	3011392323012
Matheus Prado	3011392323031
Matheus Fernandes	3011392323014
Fabricio Soica	3011392323002

Curso: Tecnologia em Desenvolvimento de Software Multiplataforma

Semestre/Ano: 6º Semestre / 2026

Professor(es) Orientador(es): Prof. Dr. Cassio R. F. Riedo

1.3. Data da Elaboração: 28/04/2026

1.4. Versão do Documento: 2.0 — 08/06/2026

2. Visão Geral do Produto

2.1. Problema a ser Resolvido

Problemas identificados no PBB Canvas FoodClub v2 (ver Seção 13.2):

#	Problema
P1	Sem testes automatizados no projeto
P2	Pipeline CI/CD sem testes automatizados
P3	Cardápio sem busca semântica
P4	Atendimento ao usuário é manual (sem suporte automatizado)
P5	Gestão manual de pedidos (sem automação)
P6	Sem dashboards de gestão para empresa e restaurante

2.2. Solução Proposta

Cada problema está associado a pelo menos uma expectativa:

Problema	Expectativa / Solução
P1	Cobertura de testes automatizados $\geq$ 80% (Jest)
P2	Pipeline CI/CD ativo com execução automática de testes a cada push
P3	Busca por voz e semântica de restaurantes/pratos no app mobile
P4	Chatbot com PLN para suporte automático a perguntas frequentes
P5	Pedidos automatizados com seleção semanal recorrente pelo funcionário
P6	Dashboards de gestão para empresa (acompanhar pedidos) e restaurante (painel de pedidos e avaliações)

O FoodClub é uma plataforma mobile B2B que conecta restaurantes, empresas e funcionários em um único ecossistema digital, com API RESTful, autenticação JWT, deploy em nuvem Azure, módulo de PLN (busca por voz + chatbot) e cobertura de testes automatizados.

2.3. Público-Alvo (Personas)

Restaurante Estabelecimento parceiro que precisa expandir sua clientela B2B além do balcão. Atualmente perde pedidos por falta de um canal digital direto com empresas. No FoodClub,

cadastra seu cardápio, define preços, recebe pedidos corporativos e atualiza o status de preparo em tempo real.

Empresa Organização de pequeno ou médio porte (10–500 funcionários) que hoje gerencia o benefício de alimentação com vales em papel ou planilhas. No FoodClub, vincula-se a um restaurante parceiro, cadastra seus funcionários e acompanha todos os pedidos em um único lugar.

Funcionário Colaborador da empresa cadastrada que hoje perde tempo saindo do escritório para buscar refeição. No FoodClub, acessa pelo celular, escolhe os pratos de cada dia da semana e recebe sua refeição sem precisar se locomover.

3. Requisitos Funcionais e Não Funcionais

3.1. Requisitos Funcionais (RFs)

Cada RF está associado a pelo menos uma persona.

ID	Funcionalidade	Persona(s)
RF01	Realizar cadastro de restaurante	Restaurante
RF02	Realizar cadastro de empresa (fluxo multi-step: conta, dados empresariais, endereço)	Empresa
RF03	Realizar login com email e senha	Restaurante, Empresa, Funcionário
RF04	Recuperar senha	Restaurante, Empresa
RF05	Cadastrar e gerenciar funcionários	Empresa
RF06	Cadastrar pratos no cardápio (nome, descrição, preço, foto)	Restaurante
RF07	Exibir lista de restaurantes disponíveis com filtro por nome	Empresa, Funcionário
RF08	Buscar restaurante por voz (expo-speech-recognition)	Empresa, Funcionário
RF09	Visualizar detalhes e cardápio do restaurante	Empresa, Funcionário
RF10	Selecionar pratos por dia da semana (pedido semanal recorrente)	Funcionário
RF11	Realizar pedido corporativo	Empresa
RF12	Acompanhar status do pedido em tempo real	Restaurante, Empresa, Funcionário
RF13	Atualizar status do pedido (pending -> confirmed -> preparing -> delivered)	Restaurante
RF14	Avaliar restaurante (1–5 estrelas + comentário)	Empresa, Funcionário
RF15	Avaliar prato (1–5 estrelas)	Funcionário

ID	Funcionalidade	Persona(s)
RF16	Editar perfil de usuário (foto, senha)	Restaurante, Empresa, Funcionário
RF17	Chatbot FAQ para suporte ao usuário (respostas a perguntas frequentes)	Restaurante, Empresa, Funcionário

3.2. Requisitos Não Funcionais (RNFs)

Cada RF crítico está associado a pelo menos um RNF.

ID	Requisito	Critério de Aceitação	RFs Relacionados
RNF01	Desempenho	Endpoints críticos respondem em < 2 s para 90% das requisições sob carga normal	RF03, RF07, RF12
RNF02	Segurança — Autenticação	Todas as rotas protegidas exigem Bearer JWT válido; token expira em 24 h	RF03
RNF03	Segurança — Dados sensíveis	Senhas armazenadas com bcrypt; CPF/CNPJ não expostos em respostas da API	RF01, RF02, RF05
RNF04	Disponibilidade	Aplicação disponível 99% do tempo em horário comercial	RF03–RF17
RNF05	Escalabilidade	Sistema suporta 200 usuários simultâneos sem degradação de performance	RF07, RF12
RNF06	Usabilidade	Interface responsiva; busca por voz funcional em Android 5+ e iOS 13+	RF08, RF17
RNF07	Manutenibilidade	Cobertura de testes unitários >= 80% nos módulos user, company-order e dish	RF01–RF17
RNF08	Compatibilidade	App funcional em Android 10+ e iOS 14+; API stateless e versionada	RF01–RF17

4. Arquitetura do Sistema

4.1. Visão Geral da Arquitetura

O FoodClub adota arquitetura **cliente-servidor** com separação clara entre frontend mobile e backend REST. O backend segue **Clean Architecture** com quatro camadas (Interfaces, Aplicação, Domínio, Infraestrutura), o que facilita testes unitários e substituição de tecnologias. A comunicação é via HTTPS com tokens JWT.

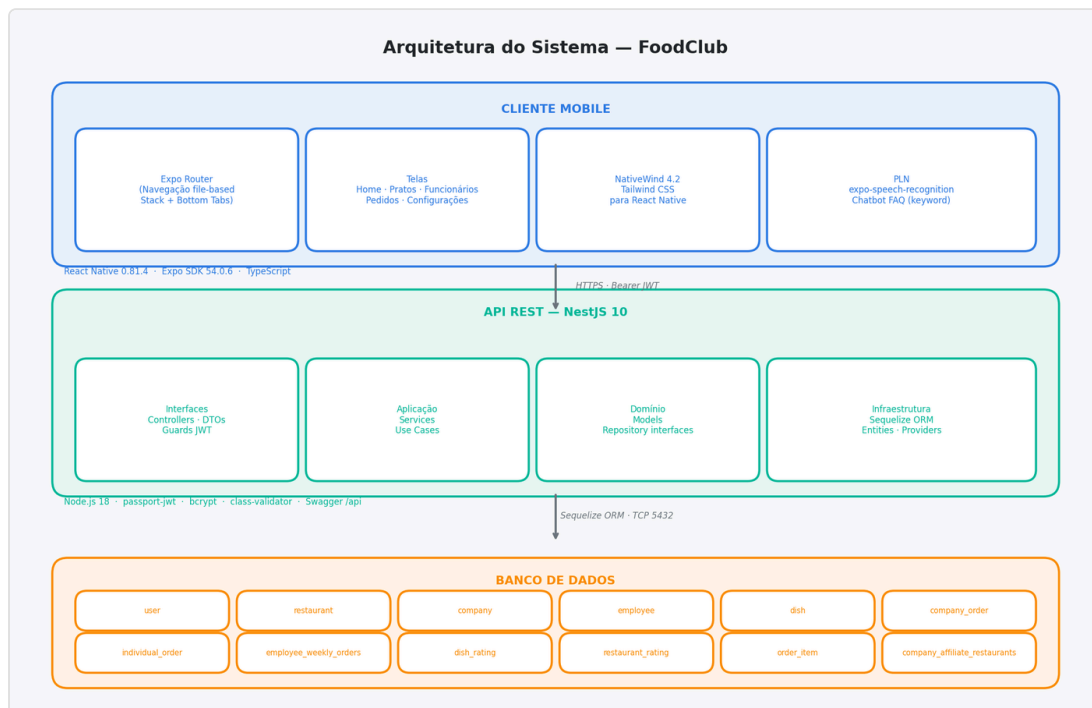


Diagrama de Arquitetura do Sistema

4.2. Componentes Principais

Componente	Tecnologia	Versão	Responsabilidade
App Mobile	React Native	0.81.4	Interface do usuário mobile (Android e iOS)
Roteamento	Expo Router	6.0.3	Navegação file-based + Stack + Bottom Tabs
Estilo	NativeWind	4.2.0	Tailwind CSS adaptado para React Native
Cliente HTTP	Axios	1.x	Chamadas REST com interceptors JWT
API Backend	NestJS	10.0.0	Servidor REST com arquitetura limpa
Runtime	Node.js	20.x	Ambiente de execução do backend
ORM	Sequelize + sequelize-typescript	6.37.1 / 2.1.6	Mapeamento objeto-relacional
Banco de Dados	PostgreSQL	15.x	Persistência relacional
Autenticação	passport-jwt + @nestjs/jwt	— / 10.2.0	JWT Bearer Token
Hash de senha	bcrypt	5.1.1	Criptografia de credenciais
Validação de DTOs	class-validator + class-transformer	0.14.x	Validação de entrada na API
Documentação da API	@nestjs/swagger	7.3.0	Swagger UI em /api
Containerização	Docker (multi-stage, Node 20 Alpine)	24.x	Build e execução isolados
CI/CD	GitHub Actions	—	Pipeline automático de validação e deploy
PLN — Voz	expo-speech-recognition	latest	Transcrição de voz para texto no app

Componente	Tecnologia	Versão	Responsabilidade
PLN — Chatbot	TF-IDF + SVM (TypeScript puro)	—	Respostas a perguntas frequentes

4.3. Tecnologias Utilizadas

Camada	Tecnologia	Versão
Mobile	React Native	0.81.4
Mobile	Expo SDK	~54.0.6
Mobile	TypeScript	5.x
Mobile	NativeWind	^4.2.0
Mobile	Expo Router	~6.0.3
Mobile	Axios	1.x
Mobile	@react-navigation/bottom-tabs	^7.4.0
Mobile	expo-speech-recognition	latest
Backend	Node.js	20.x
Backend	NestJS	10.0.0
Backend	TypeScript	5.x
Backend	Sequelize	6.37.1
Backend	sequelize-typescript	2.1.6
Backend	PostgreSQL (driver pg)	8.16.2
Backend	@nestjs/jwt	10.2.0
Backend	passport-jwt	4.x
Backend	bcrypt	5.1.1
Backend	class-validator	0.14.x
Backend	@nestjs/swagger	7.3.0
Infra	Docker	24.x
Infra	GitHub Actions	—
Infra	Azure Container Apps	—
Infra	Azure Database for PostgreSQL Flexible Server	—

5. Modelo de Dados

5.1. Diagrama de Entidade-Relacionamento (DER)

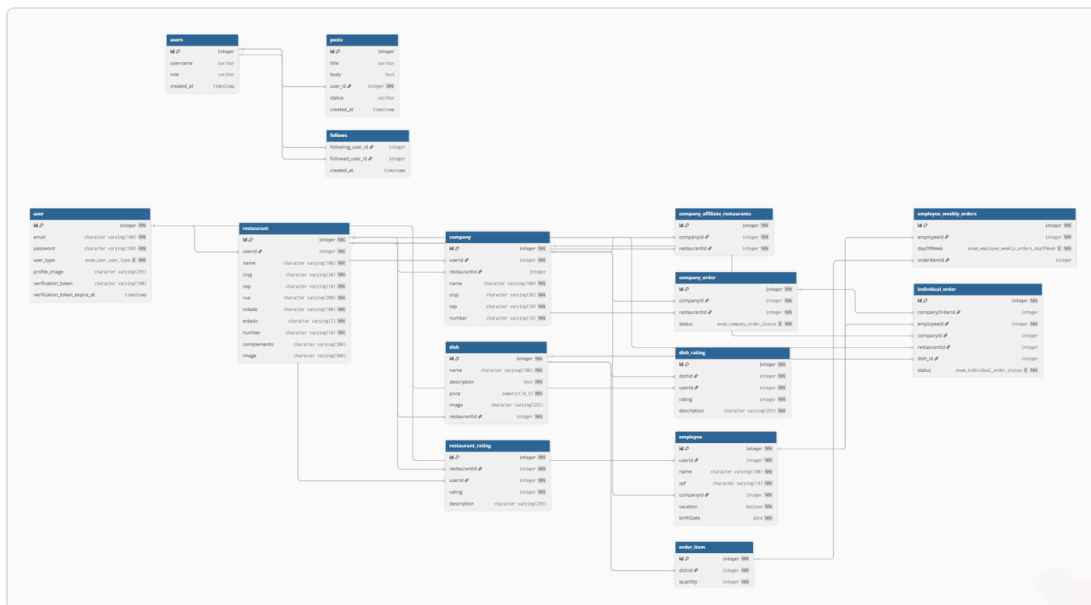


Diagrama de Entidade-Relacionamento

5.2. Descrição das Entidades Principais

user (tabela: **user**)

Atributo	Tipo	Restrição
id	INTEGER	PK, auto-increment
email	VARCHAR(100)	NOT NULL, UNIQUE
password	VARCHAR(100)	NOT NULL — hash bcrypt
userType	ENUM	‘company’, ‘employee’, ‘restaurant’
profileImage	VARCHAR(255)	nullable
phone	VARCHAR(100)	nullable
birthDate	DATE	nullable

restaurant (tabela: **restaurant**)

Atributo	Tipo	Restrição
id	INTEGER	PK
userId	INTEGER	FK -> user.id
name	VARCHAR(100)	NOT NULL
cnpj	VARCHAR(20)	
cep	VARCHAR(10)	
number	VARCHAR(10)	
rua	VARCHAR(200)	
cidade	VARCHAR(100)	
estado	VARCHAR(2)	
complemento	VARCHAR	nullable
profileImage	VARCHAR(255)	nullable

company (tabela: `company`)

Atributo	Tipo	Restrição
id	INTEGER	PK
userId	INTEGER	FK -> user.id
name	VARCHAR(100)	NOT NULL
cnpj	VARCHAR(20)	
cep	VARCHAR(10)	
number	VARCHAR(10)	
restaurantId	INTEGER	FK -> restaurant.id

employee (tabela: `employee`)

Atributo	Tipo	Restrição
id	INTEGER	PK
userId	INTEGER	FK -> user.id
companyId	INTEGER	FK -> company.id
name	VARCHAR(100)	NOT NULL
cpf	VARCHAR(14)	
birthDate	DATEONLY	
isAdmin	BOOLEAN	default false

dish (tabela: `dish`)

Atributo	Tipo	Restrição
id	INTEGER	PK
restaurantId	INTEGER	FK -> restaurant.id
name	VARCHAR(100)	NOT NULL
description	TEXT	
price	DECIMAL(10,2)	
image	VARCHAR(255)	nullable

company_order (tabela: `company_order`)

Atributo	Tipo	Restrição
id	INTEGER	PK
companyId	INTEGER	FK -> company.id
restaurantId	INTEGER	FK -> restaurant.id
status	ENUM	'pending', 'confirmed', 'preparing', 'delivered', 'canceled'

Diagrama de estados do `company_order` :

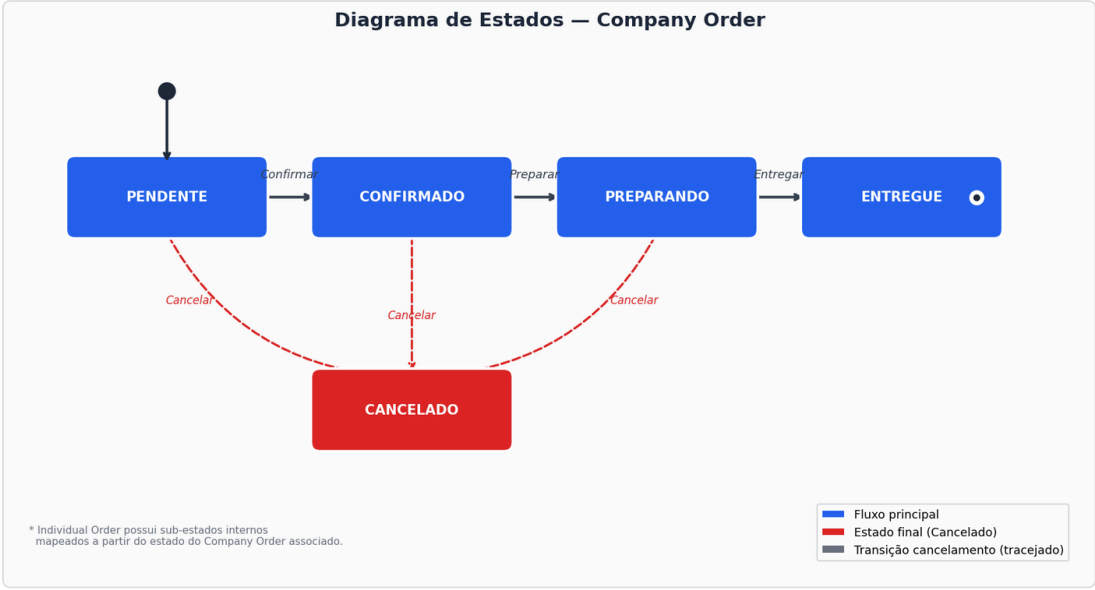


Diagrama de Estados — Company Order

individual_order (tabela: `individual_order`)

Atributo	Tipo	Restrição
id	INTEGER	PK
companyOrderId	INTEGER	FK -> company_order.id
employeeId	INTEGER	FK -> employee.id
dishId	INTEGER	FK -> dish.id
status	ENUM	'preparing', 'completed'
quantity	INTEGER	

employee_weekly_orders (tabela: `employee_weekly_orders`)

Atributo	Tipo	Restrição
id	INTEGER	PK
employeeId	INTEGER	FK -> employee.id
dayOfWeek	ENUM	'Monday', 'Tuesday', ..., 'Sunday'
orderId	INTEGER	FK -> order.id

6. APIs (Application Programming Interfaces)

6.1. Design da API

A API do FoodClub é **RESTful** com comunicação via HTTP/S e troca de dados em **JSON**. A documentação interativa está disponível em `/api` via Swagger UI quando o servidor está em execução.

Recurso	Método	Rota	Descrição
Auth	POST	/user/login	Autentica o usuário e retorna JWT + detalhes do perfil
Auth	POST	/user/logout	Invalida o token do usuário
User	GET	/user	Lista todos os usuários
User	GET	/user/:id	Retorna usuário por ID
User	POST	/user	Cria novo usuário (cadastro)
User	PUT	/user/image/:id	Atualiza foto de perfil
User	PUT	/user/password/:id	Altera senha
User	DELETE	/user/:id	Remove usuário
User	GET	/user/check-email/:email	Verifica disponibilidade de email
Restaurant	GET	/Restaurant	Lista todos os restaurantes
Restaurant	GET	/Restaurant/:id	Retorna restaurante por ID
Restaurant	POST	/Restaurant	Cadastra novo restaurante
Restaurant	PUT	/Restaurant/:id	Atualiza dados do restaurante
Restaurant	DELETE	/Restaurant/:id	Remove restaurante
Restaurant	GET	/Restaurant/:id/orders	Lista pedidos do restaurante
Restaurant	PUT	/Restaurant/orders/:id/status	Atualiza status de pedido individual

Recurso	Método	Rota	Descrição
Restaurant	PUT	/Restaurant/company-orders/:id/status	Atualiza status de pedido corporativo
Dish	GET	/Dish	Lista todos os pratos
Dish	GET	/Dish/:id	Retorna prato por ID
Dish	POST	/Dish	Cadastra novo prato
Dish	PUT	/Dish/:id	Atualiza prato
Dish	DELETE	/Dish/:id	Remove prato
Dish	GET	/Dish/by-restaurant/:restaurantId	Lista pratos de um restaurante
Company	GET	/company	Lista todas as empresas
Company	GET	/company/:id	Retorna empresa por ID
Company	POST	/company	Cadastra nova empresa
Company	PUT	/company/:id	Atualiza empresa
Company	GET	/company/:id/employees	Lista funcionários da empresa
Company	GET	/company/:id/orders	Lista pedidos da empresa
Company	POST	/company/:id/orders	Cria pedido corporativo
Company	POST	/company/:id/create-orders-from-weekly	Gera pedidos a partir do semanal
Employee	GET	/employee	Lista todos os funcionários
Employee	POST	/employee	Cadastra funcionário
Employee	PUT	/employee/:id	Atualiza funcionário

Recurso	Método	Rota	Descrição
Employee	DELETE	/employee/:id	Remove funcionário
DishRating	GET	/dish-rating/dish/:dishId	Avaliações de um prato
DishRating	POST	/dish-rating	Cria avaliação de prato
DishRating	PUT	/dish-rating/:id	Atualiza avaliação
RestaurantRating	GET	/restaurant-rating/:restaurantId	Avaliações de um restaurante
RestaurantRating	POST	/restaurant-rating	Cria avaliação de restaurante
WeeklyOrders	POST	/employee-weekly-orders	Registra prato para dia da semana
WeeklyOrders	GET	/employee-weekly-orders/employee/:id	Consulta pedido semanal
WeeklyOrders	DELETE	/employee-weekly-orders/:id	Remove seleção do dia
Health	GET	/health-check	Status da API

Códigos de resposta: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity, 500 Internal Server Error.

6.2. Autenticação e Autorização

A API utiliza **JWT (JSON Web Token)** com a biblioteca `@nestjs/jwt` + `passport-jwt`. O fluxo é:

1. Cliente envia `POST /user/login` com `{ email, password }`
2. Backend valida credenciais com `bcrypt`
3. Backend retorna `{ token: "eyJ...", userDetails: {...} }`
4. Cliente armazena o token no `authStore` (Zustand)
5. Todas as requisições subsequentes incluem `Authorization: Bearer <token>`
6. Guard JWT valida o token em cada rota protegida
7. Token expira em 24 h; em caso de 401, o app realiza logout automático (interceptor Axios em `baseRepository.ts`)

Senhas são armazenadas com hash bcrypt (salt rounds = 10). CPF e CNPJ não são expostos em respostas da API.

7. Computação em Nuvem (Azure)

7.1. Serviços Azure Utilizados

O FoodClub utiliza Azure como requisito da disciplina de Computação em Nuvem II. A infraestrutura está provisionada e operacional desde 01/05/2026:

Serviço Azure	Finalidade
Azure Container Apps	Hospedagem da API NestJS (container Docker, Node 20)
Azure Database for PostgreSQL — Flexible Server	Banco de dados gerenciado com backup automático
Azure Container Registry (ACR) — <code>acrfoodclub</code>	Armazenamento das imagens Docker da API
GitHub Actions	Pipeline CI/CD — build, test, deploy automático
Azure Key Vault — <code>kv-foodclub-bruno</code>	Gerenciamento seguro de secrets e credenciais
Managed Identity	Autenticação entre Container Apps e Key Vault (role: Key Vault Secrets User)
Azure Blob Storage	Armazenamento de imagens de perfil e cardápio (em configuração)

Estado atual: Infraestrutura Azure operacional com dois ambientes ativos (HML e PRD). Pipeline CI/CD configurado com GitHub Actions executando testes antes de cada deploy. Husky configurado para lint antes de cada commit local.

7.2. Visão Geral da Infraestrutura — Evolução AWS -> Azure

O projeto migrou da infraestrutura AWS (semestre anterior) para Azure (semestre atual), mantendo a mesma topologia lógica e adotando os serviços equivalentes da nova plataforma.

Estrutura Anterior — AWS

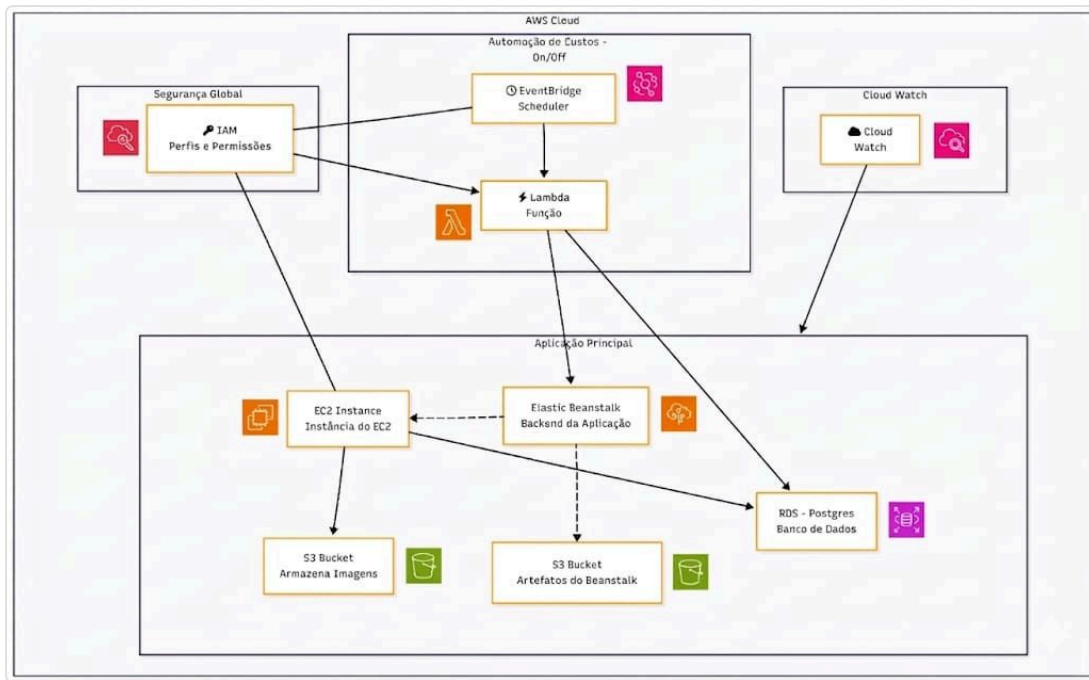


Diagrama de Infraestrutura AWS

Estrutura Atual — Azure

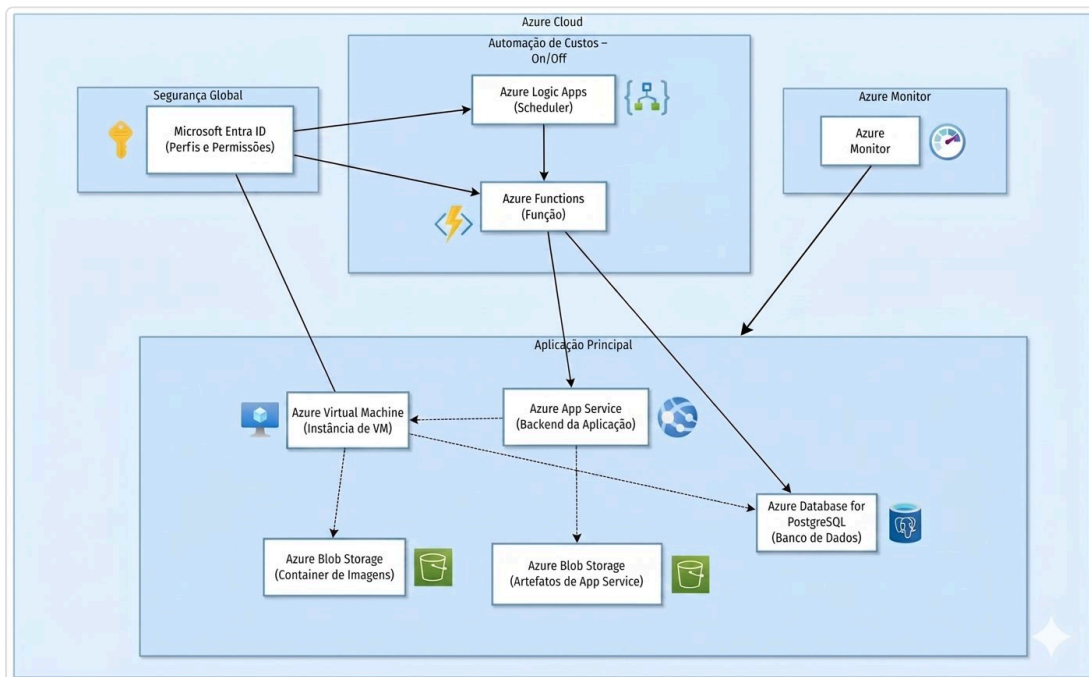


Diagrama de Infraestrutura Azure

Componente	AWS (anterior)	Azure (atual)
Segurança / IAM	AWS IAM	Microsoft Entra ID
Agendador	EventBridge Scheduler	Azure Logic Apps
Função serverless	AWS Lambda	Azure Functions
Backend da aplicação	Elastic Beanstalk	Azure Container Apps
Instância de servidor	EC2 Instance	Azure Virtual Machine
Banco de dados	RDS PostgreSQL	Azure Database for PostgreSQL
Armazenamento de objetos	S3 Bucket	Azure Blob Storage
Monitoramento	CloudWatch	Azure Monitor

7.3. Arquitetura de Deploy (Azure)

```

GitHub (push para development -> HML | push para main -> PRD)
  ↓
GitHub Actions – Pipeline CI/CD
  1. validate    -> npm ci + lint (Husky) + type check
  2. test        -> Jest (cobertura 95%)
  3. docker      -> build imagem -> push para Azure Container Registry
(acrfoodclub)
  4. deploy      -> Azure Container Apps (pull da imagem do ACR)

Ambientes:
  development -> HML: api-foodclub-hml
    URL: api-foodclub-hml.politemushroom-
    8b6971df.canadacentral.azurecontainerapps.io
  main -> PRD: api-foodclub-prod
    URL: api-foodclub-prod.politemushroom-
    8b6971df.canadacentral.azurecontainerapps.io

Banco de dados:
  Azure Database for PostgreSQL Flexible Server
  Região: Canada Central
  Segurança: SSL habilitado + firewall configurado
  Secrets: gerenciados via Azure Key Vault

```

7.4. Monitoramento e Logs

Recurso	Ferramenta	Configuração
Saúde da API	<code>GET /health-check</code> + Application Insights	Probe automático a cada 60 s
Logs da aplicação	Application Insights — Log Analytics	Retenção de 30 dias
Métricas de performance	Application Insights — Live Metrics	CPU, memória, tempo de resposta
Alertas	Application Insights — Alerts	Email em caso de p95 > 3 s ou taxa de erro > 5%
Falha no pipeline	GitHub Actions — notify job	Email automático em caso de falha

7.5. Estimativa de Custos Mensais (Azure — Brazil South)

Azure for Students: O FoodClub é um projeto acadêmico desenvolvido na FATEC. Por isso, utilizamos o **Azure for Students**, programa que concede **USD 100 de crédito gratuito por ano** sem necessidade de cartão de crédito. Toda a infraestrutura de desenvolvimento é dimensionada para operar **dentro desse crédito**, mantendo o custo real do projeto em **USD 0** durante o período acadêmico. Os valores abaixo refletem o consumo estimado dentro da cota estudantil.

Serviço	Configuração	Custo Est. (USD/mês)
Azure App Service	B1 (1 vCore, 1.75 GB RAM) — Dev/Test	~\$13
Azure Database for PostgreSQL	Burstable B1ms (1 vCore, 2 GB)	~\$13
Azure Container Registry	Basic (10 GB)	~\$5
Azure Blob Storage	LRS, < 5 GB	~\$0,10
Application Insights	< 5 GB logs/mês (free tier)	\$0
Total estimado (desenvolvimento)		~\$31/mês

Impacto do crédito estudantil: USD 31/mês representa ~31% do crédito anual (USD 100), permitindo aproximadamente **3 meses de operação contínua** sem custo. Durante sprints, os recursos são provisionados sob demanda e desalocados ao final de cada ciclo para maximizar a duração do crédito. Conversão de referência: USD 31 ~ R\$ 185/mês (câmbio 2026).

Cenário pós-acadêmico: Em produção real com tráfego efetivo, a estimativa escala para App Service P1v3 (~USD 73/mês) e PostgreSQL General Purpose (~USD 50/mês), totalizando ~USD 140/mês.

8. Qualidade e Testes de Software

8.1. Estratégia de Testes

Tipo	Etapa	Foco
Unitários	Desenvolvimento contínuo	DTOs com class-validator, Services com regras de negócio, Guards JWT
Integração	Após implementação de módulo	Endpoints REST — resposta HTTP, persistência no banco
Sistema (E2E)	Antes de cada Sprint Review	Fluxos completos de negócio (login -> pedido -> status)
Performance	Pré-deploy para produção	200 usuários simultâneos — latência e taxa de erro
Segurança	Sprint 2 em diante	Validação de token JWT, exposição de dados sensíveis

8.2. Ferramentas de Teste

Ferramenta	Versão	Finalidade
Jest	29.x	Testes unitários e de integração (backend NestJS)
Supertest	6.x	Requisições HTTP nos testes de integração
Postman / Newman	—	Testes E2E e automação de coleções de API
k6	0.50.x	Testes de performance e carga (200 VUs)
Docker Compose	—	Banco de dados isolado para o ambiente de testes

8.3. Cobertura de Testes

Camada	Meta	Resultado Atual	Status
Statements	>= 80%	95,04% (2281/2400)	Sim Atingido
Backend — módulos críticos	>= 80%	>= 95%	Sim Atingido
Frontend (Mobile)	>= 60%	—	A medir

Relatório completo: <DocumentacaoPI2026/Entregas/QualidadeSoftware/COVERAGE-REPORT-FOOD-CLUB.md>

Critérios de saída: - 100% dos casos de prioridade Alta executados - Zero defeitos críticos (bloqueadores de produção) em aberto - Teste de performance: p95 < 2 s com 200 usuários simultâneos; taxa de erro < 1%

9. Processamento de Linguagem Natural (PLN)

9.1. Funcionalidades de PLN

O FoodClub incorpora dois módulos de PLN, ambos no frontend (sem dependência de serviços de IA externos):

Funcionalidade	Descrição	Integração no produto
Busca por voz	Usuário fala o nome do restaurante; o app transcreve e filtra a lista em tempo real	Tela Home — botão de microfone ao lado da barra de busca
Chatbot FAQ	Usuário digita uma dúvida; o bot responde com base em palavras-chave predefinidas	Modal flutuante acionado por botão FAB na tela Home

9.2. Tecnologias e Modelos

Funcionalidade	Tecnologia	Abordagem
Reconhecimento de voz	<code>expo-speech-recognition</code> (Expo SDK 54)	API nativa do SO (Android SpeechRecognizer / iOS SFSpeechRecognizer) — sem modelo ML próprio
Chatbot	TF-IDF + SVM	Vetorização de texto com TF-IDF e classificação de intenções com SVM (requisito disciplina PLN — 2ª entrega)

Pipeline de processamento — Busca por voz:

```

Áudio -> expo-speech-recognition -> texto transcrito
      -> toLowerCase() -> filtro client-side sobre restaurants[]
      -> FlatList atualizada em tempo real

```

Pipeline de processamento — Chatbot FAQ:

```

Mensagem do usuário -> toLowerCase().trim() -> split(' ')
      -> tokens comparados com keywords de cada FAQ item
      -> primeira correspondência -> retorna resposta
      -> sem correspondência -> resposta padrão genérica

```

9.3. Dataset e Treinamento

O chatbot utiliza uma base de conhecimento curada com 7 categorias (pedidos, cancelamento, restaurantes, cardápio, funcionários, conta/senha, suporte), vetorizada com **TF-IDF** e

classificada com **SVM** para predição de intenção — conforme exigido na 2ª entrega da disciplina de PLN.

Item	Detalhe
Tipo	Base de conhecimento curada (pares pergunta - > intenção -> resposta)
Tamanho inicial	7 categorias, ~50 exemplos de frases por categoria
Vetorização	TF-IDF (Term Frequency–Inverse Document Frequency)
Classificador	SVM (Support Vector Machine) — classificação de intenção
Pré-processamento	Normalização (lowercase), remoção de stopwords, tokenização
Avaliação	Acurácia e F1-score por categoria sobre conjunto de validação
Reconhecimento de voz	Modelo de linguagem do SO (Google / Apple) — sem treinamento customizado

10. Integração entre Componentes

10.1. Fluxo de Dados

Formato padrão de resposta da API:

```
// Sucesso
{ "data": { ... } }

// Erro
{ "statusCode": 400, "message": "Descrição do erro", "error": "Bad Request" }
```

11. Riscos e Mitigações

11.1. Identificação de Riscos

ID	Risco	Categoria	Probabilidade	Impacto
R01	Busca por voz não funciona em Expo Go (requer EAS Dev Build)	Técnico	Alta	Médio
R02	Migração de AWS para Azure — Mitigado: Azure HML e PRD no ar desde 01/05/2026	Técnico	—	—
R03	Cobertura abaixo de 80% — Mitigado: cobertura atual de 95,04%	Qualidade	—	—
R04	Membro da equipe indisponível durante sprint crítica	Equipe	Baixa	Alto
R05	Banco de dados sem seed adequado para testes de integração	Técnico	Média	Médio
R06	Chatbot FAQ com cobertura insuficiente de perguntas	Produto	Média	Baixo
R07	Variáveis de ambiente não configuradas no ambiente de produção Azure	Infra	Baixa	Alto

11.2. Plano de Mitigação

ID	Mitigação
R01	Manter campo de texto como fallback da busca por voz; publicar EAS Dev Build para testes de voz antes da Entrega 2
R02	Mitigado — Azure deployado em 01/05/2026 com dois ambientes (HML/PRD)
R03	Mitigado — Cobertura de 95,04% atingida (meta era 80%)
R04	Documentar setup do ambiente no README; pair programming para distribuir conhecimento crítico
R05	Criar script <code>seed.ts</code> versionado no repositório e documentar no README
R06	Registrar dúvidas reais dos usuários (PO coleta via WhatsApp do grupo) e expandir <code>constants/faq.ts</code> continuamente
R07	Checklist de variáveis de ambiente no README; usar Azure Key Vault para secrets em produção

12. Gestão de Sprints e Backlog

12.1. Apreciação dos Sprints Realizados

Sprint 01 — 27/03/2026 a 10/04/2026

User Stories: US27, US28, US29, US30 | **Status:** Concluído

Reunião de Planejamento Definição do escopo de qualidade para o semestre: levantamento de requisitos, elaboração do plano de teste e definição dos papéis do time de QA. Sprint orientada pelas disciplinas de Qualidade e Testes de Software e Projeto Integrador.

O que foi entregue - 10 Requisitos Funcionais e 6 Requisitos Não Funcionais documentados - Plano de Teste com objetivo, itens testados, estratégia, critérios de entrada/saída, recursos e cronograma - Papéis e responsabilidades do processo de teste definidos para o time - Casos de teste baseados em Partição de Equivalência (técnica de caixa preta)

Impedimentos encontrados - Sprint curto (2 semanas) no início do semestre, com o time ainda em fase de organização - Escopo das entregas não estava totalmente claro na reunião de planejamento inicial - Dificuldade de alinhamento na divisão de tarefas entre os membros

Reunião de Encerramento Todas as US da sprint foram entregues. Revisão dos artefatos gerados e validação com o PO.

Lição aprendida Definir critérios de aceite detalhados antes de iniciar o sprint reduz retrabalho. A reunião de planejamento precisa de tempo suficiente para que todos os membros entendam o escopo e os critérios de cada entrega antes de começar.

Sprint 02 — 11/04/2026 a 21/04/2026

User Stories: US32, US35, US36, US55 | **Status:** Concluído

Reunião de Planejamento Definição das entregas de automação de testes e infraestrutura em nuvem. Sprint cobriu duas frentes simultâneas: implementação da suíte de testes com Jest no backend e provisionamento da infraestrutura Azure para os ambientes de homologação e produção.

O que foi entregue - Testes unitários automatizados com Jest para os principais Use Cases do sistema (Autenticação, Pedidos, Restaurantes, Empresas) - Relatório de cobertura de código: **95,04%** - Relatório de execução dos testes com todos os casos passando - Infraestrutura Azure completa: Container Apps (HML + PRD), PostgreSQL Flexible Server, Azure Container Registry, CI/CD automatizado via GitHub Actions com Managed Identity e Key Vault

Impedimentos encontrados - Sprint de 10 dias com volume alto de entregas simultâneas em frentes distintas - Dois membros (Bruno Pasqual e Matheus Prado) iniciaram a configuração da infraestrutura Azure em paralelo sem alinhamento prévio, resultando na criação de dois Resource Groups distintos

Reunião de Encerramento Todas as US foram entregues. A infraestrutura foi consolidada no Resource Group gerenciado por Bruno Pasqual, com acesso concedido a todos os membros do time. A cobertura de testes superou a meta de 80% estabelecida no plano de teste da Sprint 01.

Lição aprendida Para tarefas que envolvem infraestrutura compartilhada, é essencial comunicar e definir responsabilidades antes de executar. Alinhar quem faz o quê na planning evita retrabalho e conflito de recursos em nuvem.

Sprint 03 — 22/04/2026 a 12/05/2026

User Stories: US33, US56, US57 | **Status:** Concluído (US39 remanejada para Sprint 04)

Reunião de Planejamento Sprint planejada com foco em três frentes: implementação de busca e geolocalização de restaurantes no app mobile, integração do módulo de PLN (busca por voz), e produção da documentação técnica completa do sistema para apresentação à banca. A US39 (política de descontos) foi incluída no planejamento mas remanejada ao final da sprint por indisponibilidade de escopo.

O que foi entregue - Busca por voz integrada na tela de restaurantes via `expo-speech-recognition` (US56 — 8 SP) - Busca de restaurantes com filtros e geolocalização no app mobile (US33 — 5 SP) - Documentação técnica completa: diagramas de arquitetura, fluxo de dados, infraestrutura AWS -> Azure, seção de PLN, estimativa de custos e riscos (US57 — 3 SP) - Apresentação para a banca realizada em 04/05/2026

Impedimentos encontrados - A preparação para a banca em 04/05/2026 concentrou o esforço do time no meio da sprint, reduzindo o tempo disponível para desenvolvimento - US39 (política de descontos corporativos) não foi implementada — remanejada para Sprint 04

Reunião de Encerramento Sprint encerrada com 16 SP entregues dos 21 planejados. A US39 foi formalmente movida para o backlog da Sprint 04. A busca por voz foi validada em dispositivo real Android com `expo-speech-recognition`.

Lição aprendida Sprints que coincidem com apresentações para banca devem ter o escopo reduzido na planning para acomodar o esforço de preparação. Considerar “entrega para banca” como uma tarefa com estimativa explícita de Story Points nas próximas sprints.

Sprint 04 — 14/05/2026 a 23/05/2026

User Stories: US39, US40, US41, US42, US44, US47, US57, US58, US59, US60, US61 |
Status: Concluído

Reunião de Planejamento Planning realizada em 13/05/2026. Sprint focada em documentação (US57–US60), infraestrutura de imagens (US42/US44 backend), horários de funcionamento (US39/US40) e início do chatbot PLN (US61). US41 (avaliação de restaurante) incluída como prioridade de backend.

O que foi desenvolvido - Documentação (Alinne Martins, 13h): US57 (doc usuário), US58 (doc técnica atualizada), US59 (doc desenvolvimento), US60 (panfleto sanfona) — todos concluídos e disponíveis em `doctos/` - Backend US42 (Matheus Fernandes, 21h): validação de horário de disparo fora do expediente — PR #76 mergeado em `development` (21/05) - Backend US44 (Matheus Fernandes, 27h): edição de perfil do restaurante com troca de imagem — PR #77 mergeado em `development` (22/05) - Infra Blob Storage (Matheus Prado, 26h): migração AWS S3 → Azure Blob Storage, serviço de upload/download/delete de imagens com autenticação Azure — branch `feat/blob-storage`, PR #79 mergeado em `development` (25/05) - Backend US39 (Bruno Pasqual, 4h): horários de funcionamento do restaurante — branch `feat/opening-`

`hours`, PR pendente para Sprint 05 - Chatbot US61 (Fabricio Soica): levantamento de ferramentas iniciado — abordagem TF-IDF + SVM identificada; desenvolvimento nas próximas sprints

Resultados - US57, US58, US59, US60: **Concluídas** - US42, US44 (backend): **Concluídas** — integração mobile pendente para Sprint 05 - US41, US61: **Em andamento** — continuam no Sprint 05 - US39, US40, US47: **Não iniciadas no mobile** — carry-over para Sprint 05

Impedimentos encontrados - Frontend mobile sem progresso nas telas de horários (US39/US40) e detalhe de prato (US47) - Dependência entre backend concluído e mobile não iniciado gerou USs incompletas ao final do sprint

Lição aprendida A dependência entre backend e frontend exige sincronização mais explícita: o backend de horários e edição de perfil foram entregues sem que as telas correspondentes no mobile estivessem planejadas com o mesmo prazo. A partir do Sprint 05, cada US funcional deve ter backend e mobile planejados conjuntamente.

Sprint 05 — 24/05/2026 a 01/06/2026

User Stories: US39, US40, US41, US42, US44, US47, US61 | **Status:** Concluído

Objetivo Sprint final antes da apresentação (01/06/2026). Foco em concluir as integrações mobile pendentes, finalizar a avaliação de restaurante e evoluir o chatbot PLN.

O que foi desenvolvido - US39/US40 (Bruno Pasqual): telas mobile de horários de funcionamento e favoritos — integração com backend `feat/opening-hours` - US41: avaliação de restaurante — backend e integração mobile - US42/US44 (integração mobile): telas de validação de horário e edição de perfil do restaurante - US47 (Fabricio Soica): tela de detalhe do prato no app mobile - US61 (Fabricio Soica): início do desenvolvimento do chatbot com TF-IDF + SVM (Scikit-learn, NLTK)

Apresentação Apresentação para banca realizada em 01/06/2026. Entrega final da documentação prevista para 19/06/2026.

12.2. Itens do Product Backlog (PBI)

O backlog completo está em `DocumentacaoPI2026/backlog.md` e no GitHub Projects (iFoodClub/projects/4).

Resumo do estado atual das User Stories — **Sprint 05 encerrada (01/06/2026):**

US	Descrição	Prioridade	Status
US39	Configurar horários de funcionamento do restaurante (mobile)	Alta	Concluído
US40	Tela de restaurantes favoritos (mobile)	Média	Concluído
US41	Avaliar restaurante	Alta	Concluído
US42	Bloquear pedido fora do horário de funcionamento	Alta	Concluído
US44	Editar perfil do restaurante com troca de imagem	Alta	Concluído
US47	Tela de detalhe do prato (mobile)	Média	Concluído
US57	Documentação do Usuário (PDF)	Alta	Concluído
US58	Documentação Técnica (este documento)	Alta	Concluído
US59	Documentação de Desenvolvimento (PDF)	Alta	Concluído
US60	Panfleto Sanfona (PDF)	Média	Concluído
US61	Chatbot de sugestão de pratos (PLN)	Média	Em andamento

O backlog foi construído com base no PBB Canvas (Aguiar; Caroli, 2021) e priorizado pelo valor de negócio para os três atores: Restaurante, Empresa e Funcionário.

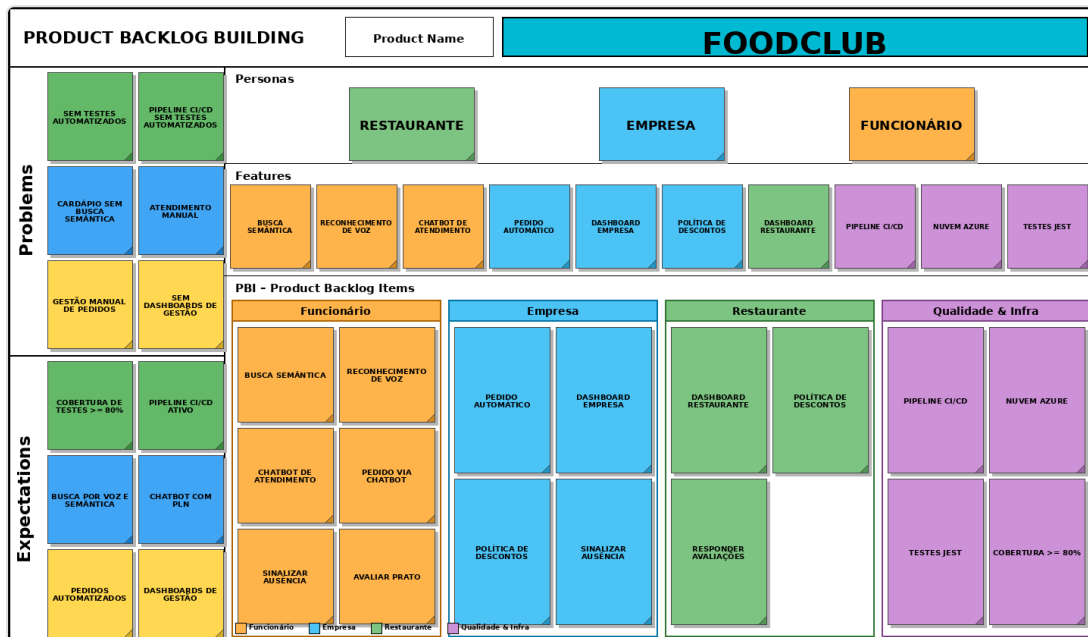
13. Anexos

13.1. Backlog do Produto

- Backlog completo: github.com/iFoodClub/projects/4 (GitHub Projects Board)
- Arquivo markdown: [DocumentacaoPI2026/backlog.md](#)
- Sprints documentadas: [DocumentacaoPI2026/sprints/](#)

13.2. Outros Diagramas

PBB Canvas — FoodClub v2



PBB Canvas FoodClub v2

- DER visual (exportar do DBeaver/pgAdmin): [doctos/der.png](#)
- Diagrama de estados do CompanyOrder: [DocumentacaoPI2026/QualidadeSoftware/item3-diagrama-estados-company-order.md](#)
- Plano de testes completo: [DocumentacaoPI2026/QualidadeSoftware/item1-plano-de-teste.md](#)
- Partição de equivalência: [DocumentacaoPI2026/QualidadeSoftware/item2-caixa-preta-particao-equivalencia.md](#)

13.3. Referências

AGUIAR, F.; CAROLI, P. **Product Backlog Building** — Um guia prático para criação e refinamento de backlog para produtos de sucesso. Rio de Janeiro: ed. Caroli, 2021.

DYNOWSKI, L.; DULAK, M. **Dominando estilos de APIs** — Compreendendo as vantagens e desvantagens dos principais estilos de APIs e escolhendo as soluções corretas. São Paulo: Novatec, 2025.

NestJS Documentation. Disponível em: docs.nestjs.com. Acesso em: 28/04/2026.

Expo Documentation. Disponível em: docs.expo.dev. Acesso em: 28/04/2026.

Microsoft Azure Pricing Calculator. Disponível em: azure.microsoft.com/pt-br/pricing/calculator. Acesso em: 28/04/2026.

Sequelize Documentation. Disponível em: sequelize.org. Acesso em: 28/04/2026.

Nota: Este documento foi elaborado com auxílio de Inteligência Artificial.